Deccan Estates

Technology Stack Guide

What was used, what each piece is responsible for, why it was chosen, and when it came into the project.

Prepared for Yashwanth Varma Konduru

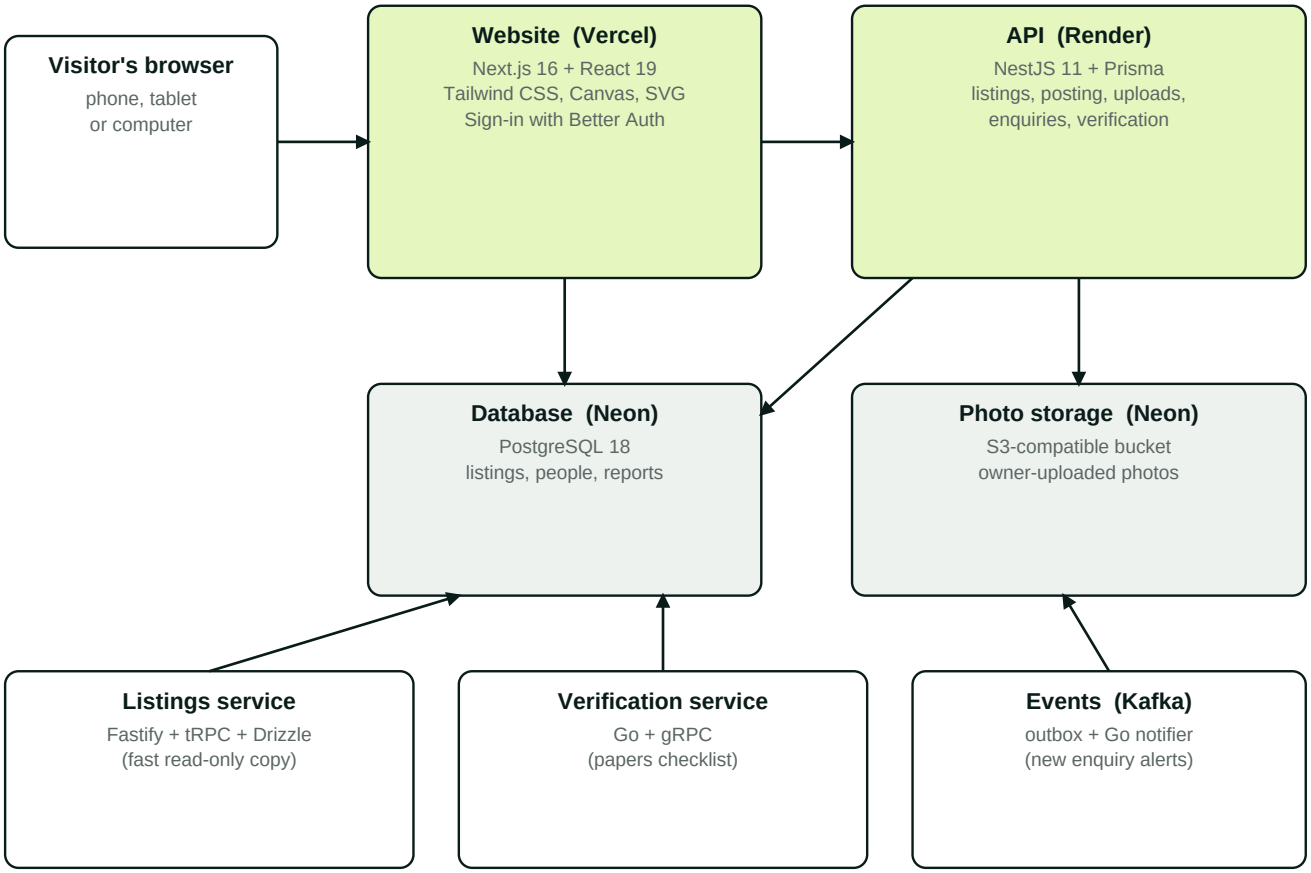
26 October 2026 | Hyderabad real-estate platform

1. The big picture

Deccan Estates is a website where people in Hyderabad search for plots, flats, villas, gated communities and commercial spaces to buy, rent or lease, and where owners and agents can post properties. Behind the pages sit a few cooperating parts. This guide explains each part in plain language, then lists every technology with what it does, why it was picked, and when it entered the project.

How the parts fit together

Arrows show who talks to whom. The three small services at the bottom are optional add-ons.



Two everyday journeys

- **Opening the listings page.** The browser asks the website (Next.js on Vercel). The website asks the API (NestJS on Render) for matching listings. The API reads them from the database (PostgreSQL on Neon), adds each listing's agent, area and verification report, and sends them back. The website draws the cards and the map.
- **Posting a property.** The person signs in (Better Auth, on the website). They upload photos, which the API checks and stores in Neon's photo bucket. They submit the form; the API validates every field with Zod, saves the listing in PostgreSQL, and the new home appears in search straight away.

Why this shape?

Each part has one job and can be replaced without rewriting the others. The website only draws things; the API owns the rules and the data; the database remembers; the storage bucket keeps photos. The smaller services (listings, verification, events) were added later to show how the system can grow without changing what visitors see.

2. When each piece came in

The project grew in stages. Stage A was already in place when this conversation began. The rest were built or changed during it, in this order. The repository history spans 21 to 26 October 2026 (56 commits).

Stage	What happened	Main technologies
A. Foundation (before this conversation)	A pnpm monorepo with the website, the NestJS API, the AI service, Postgres and Redis, sample Hyderabad listings, sign-in with email, passkeys and Google, saved homes, filters and a map.	pnpm, Next.js, React, Tailwind, NestJS, Prisma, PostgreSQL, Redis, Better Auth, MapLibre, FastAPI, pgvector, TanStack Query
B. Design and the home experience	Rebuilt the look after a dark, editorial reference design. Added the sunset-to-night-to-space opening, the scroll-driven wireframe 'build your place' story, the animated loader, the loan calculator inside a gate-and-wall frame, the skyscraper-shaped verification showcase, areas wall, FAQ and footer.	Canvas 2D, SVG, CSS variables, Tailwind v4, next/font, IntersectionObserver, requestAnimationFrame
C. Trust features	Verified Land Report: a deterministic checklist per listing with a score. Sell-with-us enquiry form with spam trap and validation.	Prisma migrations, Zod, custom report builder, Vitest
D. Search, polish and quality	Search bar on every inner page, advanced filters, map hover cards, error and loading pages, sitemap and robots, privacy and terms pages, a full bug sweep.	TanStack Query, MapLibre popups, Next metadata, ESLint, Vitest
E. Growth services (Phase 2)	Docker images, Helm chart, Terraform, OpenTelemetry. A Fastify + tRPC + Drizzle listings service. A Go gRPC verification service with a golden-file parity test. Kafka enquiry events with a transactional outbox and a Go notifier.	Docker, Helm, Terraform, OpenTelemetry, Fastify, tRPC, Drizzle, Go, gRPC, protobuf, Kafka, kafkajs, kafka-go
F. Accounts and posting	Sign up in the header, post a property to sell, rent or commercial-lease, photo upload to S3-compatible storage, my listings, greeting and account menu, profile and settings, exact-bedroom filter, real area photos with unique covers and a credits page.	Better Auth, multer, AWS SDK (S3), Prisma, Wikimedia Commons, Pillow
G. Going live	Neon for the database and photos, Render for the API, Vercel for the website. Fixes found while deploying: pnpm build rules, Tailwind through PostCSS, optional Redis, port handling.	Neon, Render, Vercel, GitHub, pnpm allowBuilds, @tailwindcss/postcss

3. Every technology, layer by layer

Version numbers are the ones in the project at the time of writing. 'Responsible for' describes the job the technology does here, not everything it can do.

3.1 Languages, tools and workflow

Technology	What it is responsible for	Why we chose it	When it was used
TypeScript 5	Almost all of the website, the API and the listings service. Catches mistakes before the code runs.	One language across front and back end; the compiler checks data shapes between them.	Foundation, every stage
Node.js 22	Runs the API, the listings service, the build and the tests.	Current long-term-support version; supported by Vercel and Render.	Foundation
pnpm 12 workspaces	Installs packages once for the whole repository (apps/web, services/*).	Fast, disk-efficient, strict about dependencies. Needed explicit 'allowBuilds' rules for the server build.	Foundation; build rules in Stage G

Technology	What it is responsible for	Why we chose it	When it was used
Git and GitHub	Keeps every change, and feeds Vercel and Render.	Standard; both hosts deploy straight from a repository.	All stages; private repo created in Stage G
ESLint 9, Prettier	ESLint enforces code rules; Prettier is configured for formatting.	Stops bugs such as state updates inside effects.	Foundation; ESLint run before each commit
Go 1.27, buf, protoc plugins	Compile the verification and notifier services and generate gRPC code.	Small fast binaries; the right tool for service-to-service calls.	Stage E

3.2 The website (what visitors see)

Technology	What it is responsible for	Why we chose it	When it was used
Next.js 16.4 (App Router, Cache Components, Turbopack)	Builds pages, routes, metadata, sitemap, error pages, image resizing, and caches listing pages for minutes or hours.	Fast first load through server-rendered pages, good for search engines, one codebase for pages and small server routes.	Foundation; reworked in B, D and G
React 19.3	All interface components, from cards to the calculator.	Industry standard; Next.js is built on it; lets scroll animations be written as small components.	Foundation
Tailwind CSS 4 (via PostCSS)	All styling: colors, spacing, responsive layouts, dark theme.	Fast to change, consistent design tokens. Moved from a custom Turbopack loader to the standard PostCSS plugin to fix the Vercel build.	Foundation; PostCSS switch in Stage G
next/font: Inter Tight and Geist Mono	Headline and body text; the capital-letter button and label style.	Self-hosted at build time, so no flicker and no extra request to Google while browsing.	Stage B
Canvas 2D and SVG (hand-written)	The sunset villa, night sky, stars, black hole, wireframe plot, villa, gated community and tower, wisp ribbons, and the skyscraper.	Gives full control and smooth scroll-driven motion without a heavy 3D library; a first 3D attempt was removed.	Stage B
TanStack Query 5	Fetches and caches data in the browser (map points, saved homes, my listings).	Handles loading, errors and refetching so components stay simple.	Foundation; extended in D and F
Zustand 5	Small shared state, such as the compare list.	Tiny and simple compared with larger state tools.	Foundation
Zod 4	Checks every response from the API and every form in the browser and on the server.	One schema gives both a type and a runtime check.	Foundation; used heavily in C and F
MapLibre GL 5 + OpenStreetMap tiles	The results map: pins, clusters, search-area outline and hover cards.	Open and free; no map key required.	Foundation; hover cards in Stage D
lucide-react, class-variance-authority, clsx, tailwind-merge	Icons, and tidy combining of style classes.	Light and consistent.	Foundation

3.3 Sign-in and accounts

Technology	What it is responsible for	Why we chose it	When it was used
Better Auth 1.7	Sign up, log in, sessions, password change, and short-lived tokens for the API.	Runs inside the website, no separate login server; supports passkeys and Google. Stores its tables in a separate 'auth' schema so database migrations never touch them.	Foundation; menu, profile and settings in Stage F
Passkeys (@better-auth/passkey)	Sign in with fingerprint, face or device PIN.	More secure than passwords and easy for visitors.	Foundation; untested live
JWT and jose 6	The website issues a 15-minute token; the API verifies it against the website's public keys.	Lets the API trust a signed-in person without sharing a session store.	Foundation
pg (node-postgres)	Lets Better Auth read and write its tables.	Standard Postgres driver.	Foundation

3.4 The API (the rules and the data)

Technology	What it is responsible for	Why we chose it	When it was used
NestJS 11	Receives requests, applies rules, returns listings, saved homes, enquiries, uploads and my-listings actions.	Clear structure (modules, controllers, services), built-in validation hooks, easy to test.	Foundation
Prisma 6 + migrations	Describes the database in one file, creates tables, and runs queries safely.	Type-safe queries and a reviewable migration history. Eight migrations so far, including verification reports, enquiries, outbox events, deal type and owners.	Foundation; extended in C, E, F
PostgreSQL 18 on Neon	Stores listings, areas, agents, verification reports, enquiries, saved homes, outbox events and sign-in tables.	Reliable relational database; Neon offers a free hosted tier with branching.	Foundation locally; Neon in Stage G
Redis 7 + ioredis 6 (optional)	Caches listing pages and enforces rate limits and posting limits.	Speeds up repeated searches. Now optional: REDIS_URL=off runs without it for the free setup.	Foundation; made optional in G
Zod validation	Rejects bad input: odd filters, short titles, fake photo names, bad phone numbers.	Fails early with clear messages.	Foundation; posting rules in F
@nestjs/swagger	Generates interactive API documentation.	Quick way to see and try every route.	Foundation
multer + image sniffing	Receives photo uploads and checks the real file type from its first bytes (JPEG, PNG, WebP; max 5 MB).	Never trusts a file name or browser-supplied type; stops disguised scripts.	Stage F
@aws-sdk/client-s3	Writes and deletes photos in an S3-compatible bucket (Neon storage). Falls back to local disk.	One code path works for Neon, Cloudflare R2, Supabase or Amazon S3.	Stage F; used live in G
Rate limit guard	Limits each visitor to 120 requests a minute (when Redis is on).	Protects the API from abuse.	Foundation

3.5 Growth services (Phase 2)

Technology	What it is responsible for	Why we chose it	When it was used
Fastify 5 + tRPC 11 + Drizzle ORM	A read-only listings service with the same answers as the API. The website can use it and falls back to the API.	Shows a faster, lighter way to serve searches. A parity test compares it with the API across many filters.	Stage E
Go + gRPC + Protocol Buffers	Builds the verification report for a listing. The API calls it and falls back to its own code if it is down.	Typed contract between services; Go is quick and small. A golden test checks 1,200 reports match the original TypeScript.	Stage E
Kafka + kafkajs + kafka-go	Carries 'new enquiry' events from the API to a notifier that logs or posts a summary to a webhook.	Lets other things react to enquiries later without changing the form.	Stage E (not run against a real broker yet)
Transactional outbox	Saves the enquiry and its event in one database transaction, then publishes and retries.	An event is never lost if Kafka is down; delivery is at least once.	Stage E
OpenTelemetry	Sends request traces to a collector when one is configured.	Standard way to see slow calls across services.	Stage E

3.6 The AI assistant

Technology	What it is responsible for	Why we chose it	When it was used
Vercel AI SDK (ai 7, @ai-sdk/anthropic, @ai-sdk/openai, @ai-sdk/react)	Chat on each listing page, with streamed answers; picks a model provider.	One interface for several model providers.	Foundation
FastAPI + pgvector + fastembed + LlamaIndex	Reads property documents, turns them into searchable vectors and retrieves passages (retrieval-augmented answers).	Python has the best tools for document processing; embeddings run locally, so no data leaves.	Foundation; not retested in this conversation
Prompt-injection guard	Treats document text as data, not instructions.	Documents may contain hostile text.	Foundation

3.7 Photos and content

Technology	What it is responsible for	Why we chose it	When it was used
Wikimedia Commons (CC BY, CC BY-SA, CC0)	Real photos of Hyderabad areas, matched to the kind of property. 127 photos, each with author and licence on the /credits page.	Free to use with credit; far better than random stock images. Coverage is thin for some areas, so some covers are crops of larger photos.	Stage F
Pillow (Python)	Resized, cropped and compressed the photos to about 48 MB in total (127 files).	Simple and reliable image processing.	Stage F (one-time)
next/image	Serves each photo in a sensible size and in AVIF or WebP.	Smaller downloads, faster pages.	Foundation; extended in F

3.8 Hosting and deployment

Technology	What it is responsible for	Why we chose it	When it was used
Vercel	Hosts the website.	Built for Next.js; free for testing. Its free plan is for non-commercial use.	Stage G
Render	Hosts the API on a free web service.	Simple deploys from a repository; sleeps when idle on the free plan.	Stage G

Technology	What it is responsible for	Why we chose it	When it was used
Neon	Hosts PostgreSQL 18 and the photo bucket in Singapore.	Free tier, branching, S3-compatible storage; near Hyderabad.	Stage G
Docker + Docker Compose	Packages the website, API and services; runs them together with Postgres, Redis and Kafka.	Same setup anywhere.	Stage E (images not built here)
Helm chart, Terraform (AWS)	Describe a Kubernetes deployment and AWS Postgres and Redis.	For a later move to a large cloud setup.	Stage E (not applied yet)
render.yaml	Tells Render how to build and start the API.	Repeatable setup; secrets are asked for, not stored.	Stage G

3.9 Testing and checking

Technology	What it is responsible for	Why we chose it	When it was used
Vitest 5, Testing Library, jsdom, MSW	Unit and component tests for the website, API and listings service.	Fast and works with TypeScript out of the box.	All stages
supertest, golden-file tests, Go test	API checks; a golden file proves the Go verification matches the original; Go tests for gRPC and the notifier.	Proves two implementations agree.	Stages C and E
Playwright	End-to-end browser tests (configured).	Tests real flows.	Foundation
Overflow sweep script	Loads each page in hidden frames at 8 widths (360 to 1920 px) and reports anything outside its box.	Found the navbar, listing-page and area-name overflows.	Stage G

4. What the database remembers

Table	Holds	Notes
Locality	The 10 areas (name, zone, average price per sq ft, location).	Gachibowli, Kokapet, Kondapur, Miyapur, Narsingi, Tellapur, Shamshabad, Kompally, Adibatla, Uppal.
Agent	The 4 sample agents.	All show one phone number; they are sample data.
Listing	Every property: type (plot, villa, apartment, gated community, commercial), deal (sale, rent, lease), price, sizes, photos, location, owner or agent.	Rent and lease prices are monthly, with a deposit. Owner posts keep the poster's account id and phone private.
VerificationReport	Score, status (verified, caution, risk) and the individual checks.	Marked as demo data.
Enquiry	Leads from the Sell form.	Contact details stay here; the event carries only an id.
OutboxEvent	Events waiting to be published.	Replayed until delivered; cleaned after 7 days.
SavedListing	Which signed-in person saved which home.	Linked by the sign-in id.
auth.* (6 tables)	Accounts, sessions, passkeys, login keys, verification codes.	Kept in a separate 'auth' schema.

5. Key decisions and their reasons

- **Monorepo.** One repository for the website, API and services keeps shared changes together and CI simple (decision record 0001).
- **Sign-in lives in the website.** The API checks a token instead of running its own login, so there is one place for accounts (record 0002).
- **Own animation code, no 3D engine.** It loads faster and can be tuned to scroll position; an earlier 3D version was removed.
- **Deterministic verification reports.** The same listing always gets the same report, which makes the feature testable until real document checks exist (record 0007).
- **Services beside the API, not instead of it.** The listings and verification services run alongside the NestJS API with fallbacks, so a failure in one never takes the site down (records 0008 and 0009).
- **Outbox for events.** An enquiry is never lost because a message broker is down (record 0010).
- **Photos in object storage.** Free web hosts erase their disks when they restart, so uploads go to a bucket.
- **Optional cache.** The free setup runs without Redis; the site still works, only a little slower.

6. Security and privacy measures

- Secrets are never stored in the repository. They are entered in the host dashboards; a scan found no keys or .env files in git.
- Uploads are checked by their real file type and size, renamed to random names, and only those names are accepted when posting.
- A poster's phone number is hidden from the public API. Only signed-in people can reveal it through a separate protected call.
- Each person can post a limited number of listings per hour and have a limited number live at once, and only the owner can pause, close or delete a listing.

- Forms include a hidden spam trap, strict validation, and submit buttons that stay disabled until the page is ready.
- Sign-in redirects only accept paths on the same site, so a crafted link cannot send people elsewhere.
- Events carry an id and non-personal facts, not names or phone numbers.

7. What is real, and what is not yet proven

Area	Status
Website, API, search, map, posting, accounts, uploads, calculator, animations	Built and tested here; checked in a browser during development.
Live deployment (Neon, Render, Vercel)	Neon and the Render API are live and answer correctly. The Vercel deployment was being finalised when this guide was written.
Listings and verification reports	Sample data, clearly marked. Replace with real listings and real document checks before launch.
Kafka events, Docker images, Helm, Terraform	Written and unit tested, but not run against a real broker, builder, cluster or cloud.
AI assistant and passkeys	Present from the foundation stage; not retested live in this conversation.
Photos	Credited and licensed, but a few villa covers are crops. Real listing photos should replace them.
Legal pages	Plain-language drafts. Have a qualified person review them.
Vercel free plan	For non-commercial use; use a paid plan for a business site.

8. Plain-language glossary

Term	Meaning
API	The server that the website asks for data and actions.
Database	Where information is kept permanently, in tables.
Migration	A saved, reviewable change to the database's structure.
Cache	A quick copy of recent answers so repeated searches are faster.
JWT	A short-lived signed pass that proves who is signed in.
gRPC / tRPC	Typed ways for one program to call another.
Kafka and outbox	A message line between programs, and a safe way to make sure no message is lost.
S3-compatible storage	A file store that speaks Amazon's standard protocol, so different providers work the same way.
Docker, Helm, Terraform	Tools that package an app, run it on many machines, and describe cloud servers as code.
Turbopack / PostCSS	The tool that builds the website, and the plug-in system that turns Tailwind classes into CSS.
Parity test / golden file	A test that proves a new copy of a feature gives exactly the same answers as the original.